

BCC1313
ALGORITHMS AND COMPLEXITY

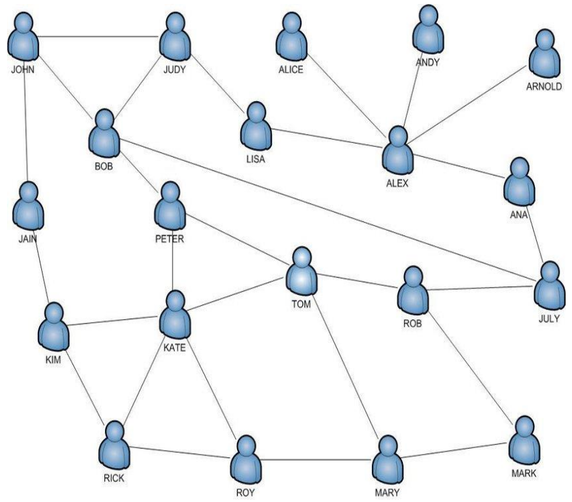
Graph Exploration

INTRODUCTION

- **Graph** is a collection of nodes, also called vertices, connected by edges.
- They represent complex relationships between entities, where each node can be connected to multiple other nodes.

APPLICATIONS OF GRAPHS

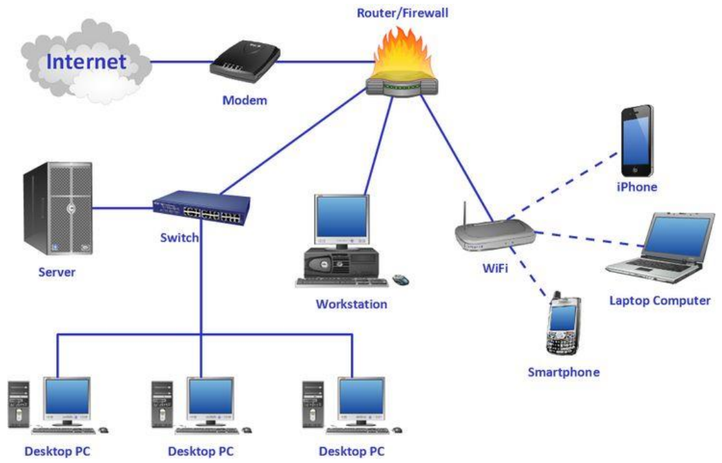
- Graphs are foundational in many real-world applications, including:
  - **Social Networks:** Users are represented as nodes, and friendships or followers as edges.
  - **Navigation and Maps:** Cities or locations are nodes, with roads or paths as weighted edges (where weights may represent distance or time).





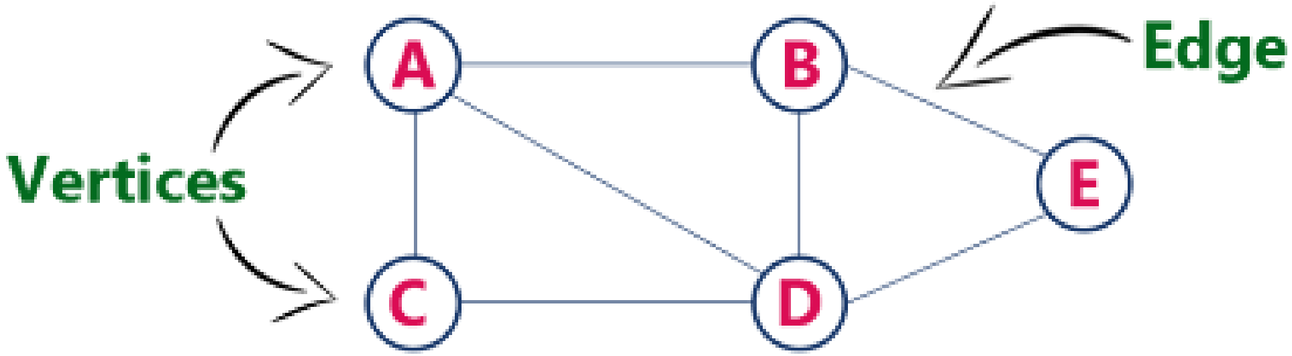
APPLICATIONS OF GRAPHS

- **Computer Networks:** Routers and devices are nodes, with communication links as edges, modeling data flow and connectivity.
- **Recommendation Systems:** Used in platforms like Netflix, Amazon, and Spotify to suggest movies, products, or music based on user preferences and relationships.



KEY COMPONENTS

- Graph components:
  - **Vertex (Node):** A point in the graph representing an entity or object.
  - **Edge (Link):** A connection between two vertices that represents a relationship or pathway between them.

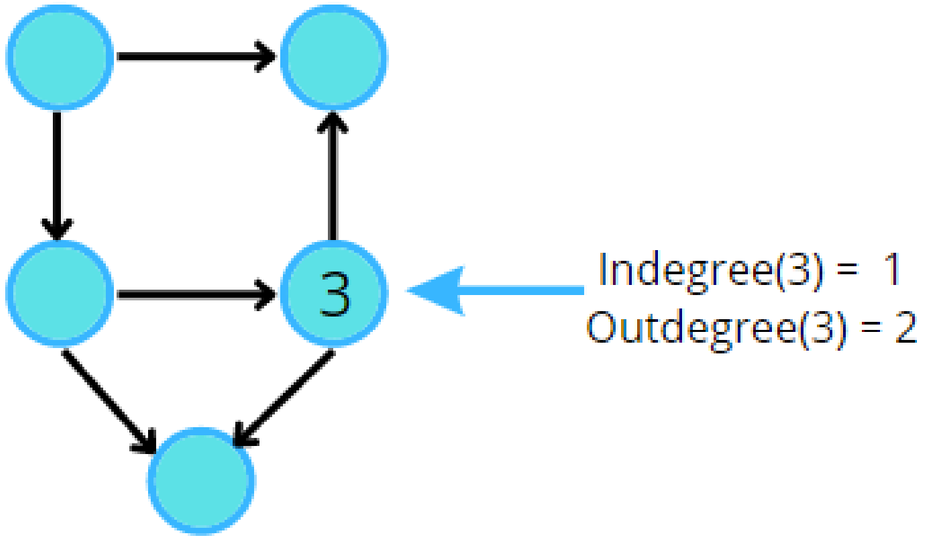


TERMINOLOGY

- **Degree:** The number of edges connected to a vertex.

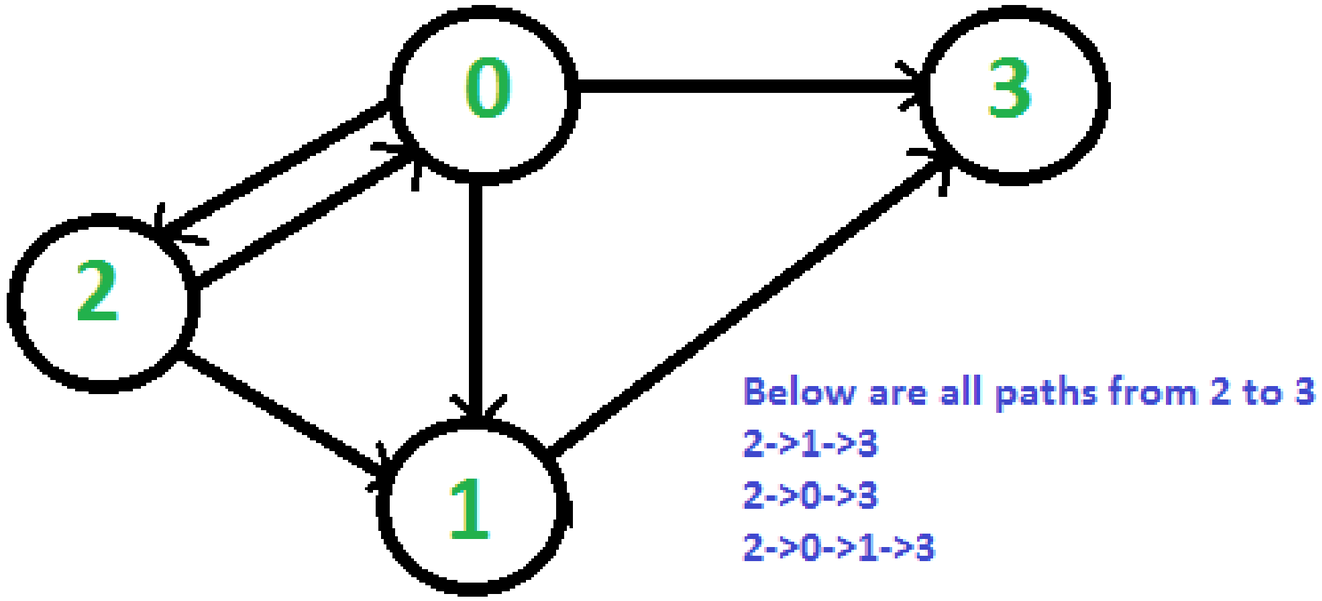
  In directed graphs, each vertex has an **in-degree (incoming edges)**

  and **out-degree (outgoing edges).**



TERMINOLOGY

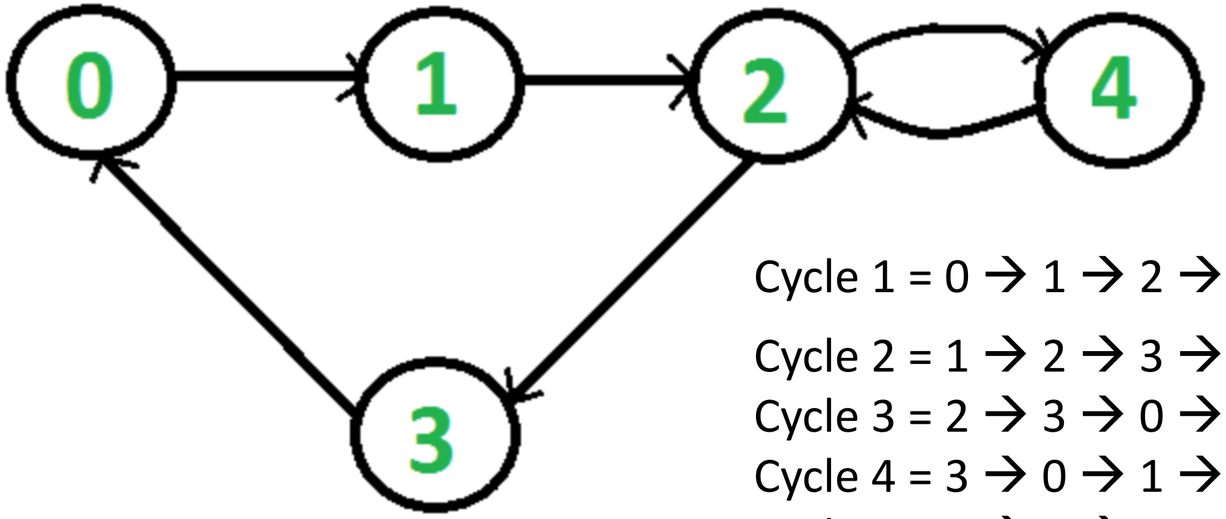
- **Path:** A sequence of edges connecting a series of vertices.



TERMINOLOGY

- **Cycle:** A path that starts and ends at the same vertex, with all vertices in between distinct.

- **Simple Cycle:** A cycle that does not pass through a vertex more than once.



Cycle 1 = 0 → 1 → 2 → 3 → 0

Cycle 2 = 1 → 2 → 3 → 0 → 1

Cycle 3 = 2 → 3 → 0 → 1 → 2

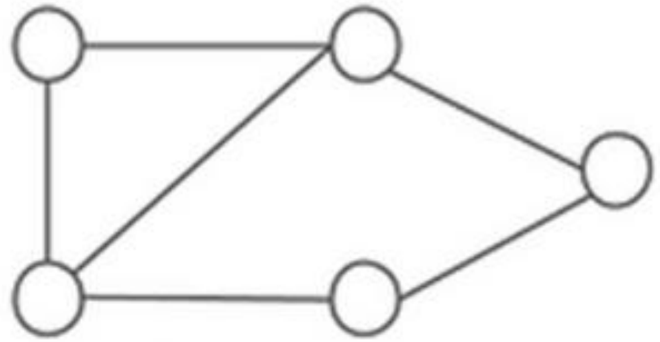
Cycle 4 = 3 → 0 → 1 → 2 → 3

Cycle 5 = 2 → 4 → 2

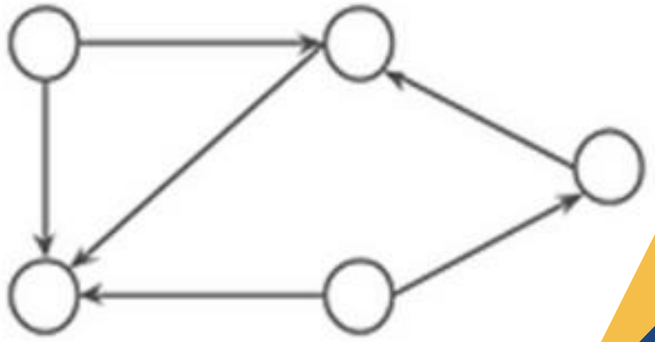
Cycle 6 = 4 → 2 → 4

TYPES OF GRAPHS

- **Undirected Graph:** Edges have no direction, meaning if there's an edge between vertices A and B, you can travel from A to B and vice versa.
- **Directed Graph (Digraph):** Edges have a specific direction, indicating a one-way relationship from one vertex to another.



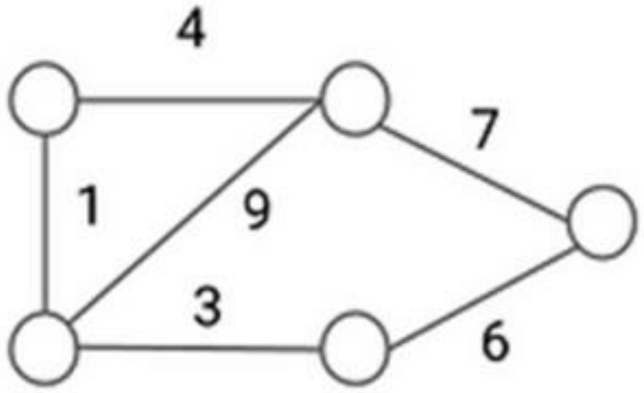
Undirected



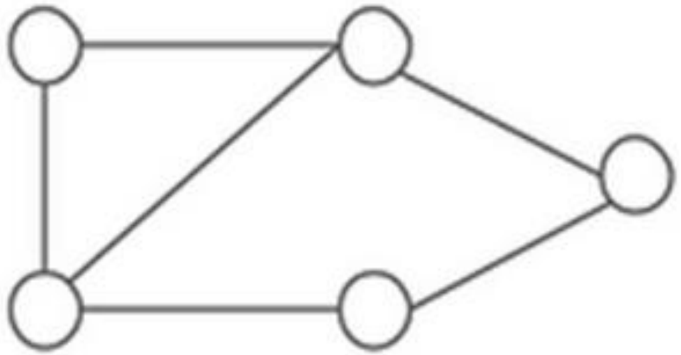
Directed

TYPES OF GRAPHS

- **Weighted Graph:** Edges have weights or costs associated with them, commonly used to represent distances, times, or costs.
- **Unweighted Graph:** Edges do not have any weights.

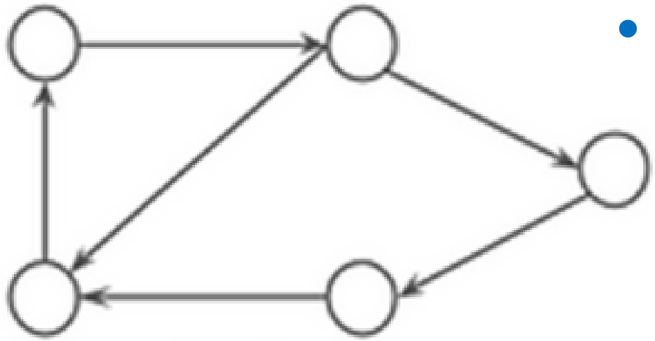


Weighted


Unweighted

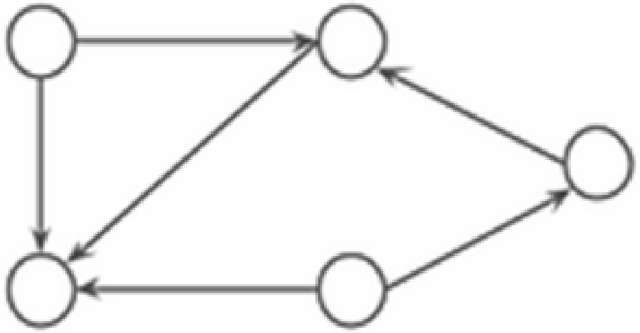
TYPES OF GRAPHS

- **Cyclic Graph:** Contains at least one cycle, where a path exists that starts and ends at the same vertex.



Cyclic

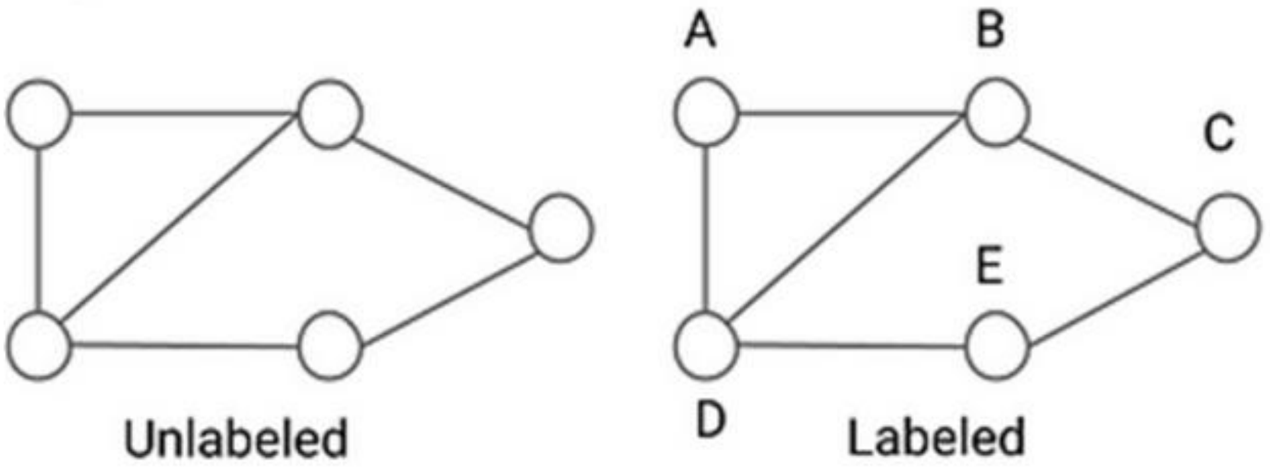
- **Acyclic Graph:** Has no cycles. A Directed Acyclic Graph (DAG) is commonly used in scheduling tasks, project management, and dependency resolution.



Acyclic

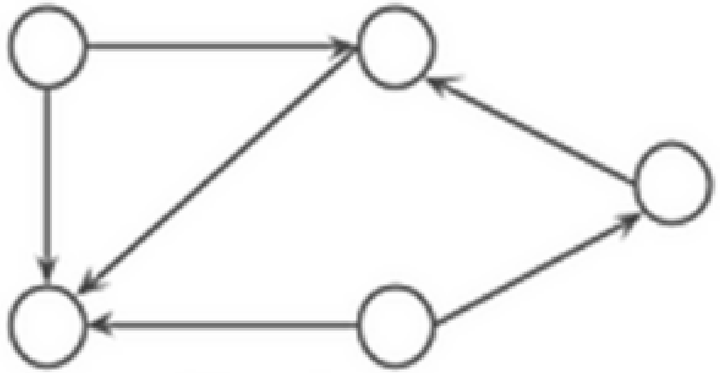
TYPES OF GRAPHS

- **Labeled Graph:** A graph where vertices and/or edges are assigned unique labels (names, numbers, weights, or attributes).
- **Unlabeled Graph:** is a graph where vertices and edges do not have specific names or values - only the structure matters.



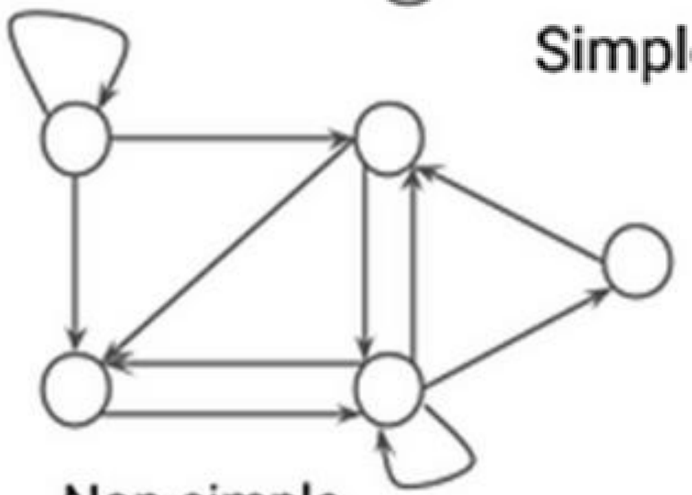
TYPES OF GRAPHS

- **Simple Graph:** A graph that has no loops (edges that start and end at the same vertex).



Simple

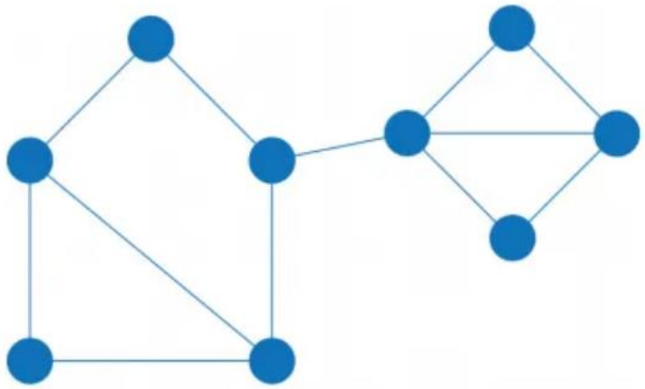
- **Non-simple Graph:** A graph that has loops and multiple edges (more than one edge between the same two vertices).



Non-simple

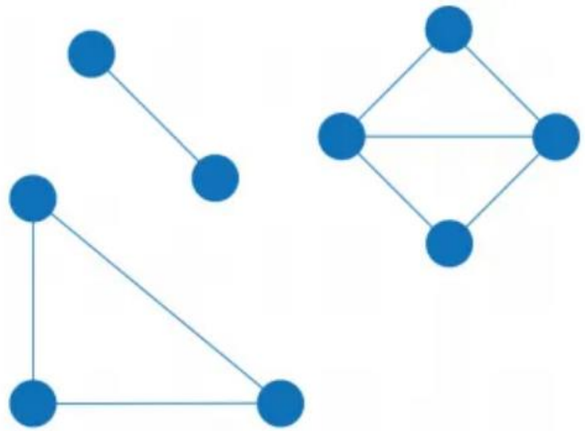
TYPES OF GRAPHS

- **Connected Graph:** A graph that has a path between each pair of distinct vertices



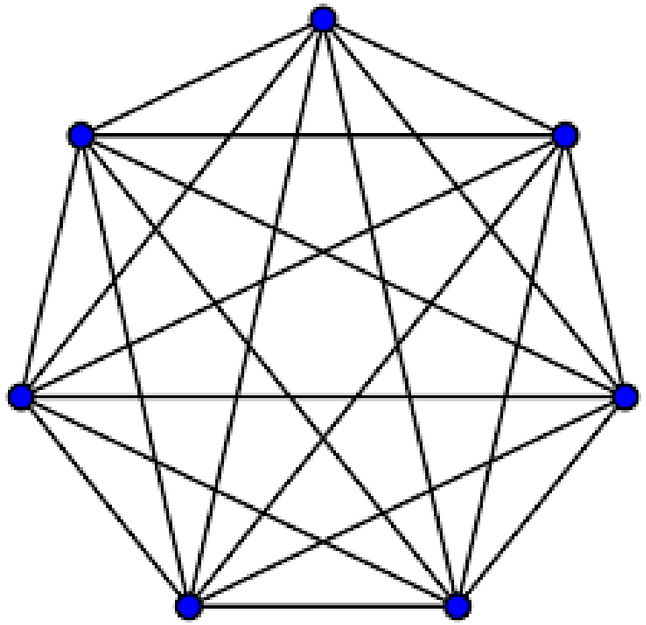
Connected Graph

- **Disconnected Graph:** A graph that has at least one pair of vertices without a path between them



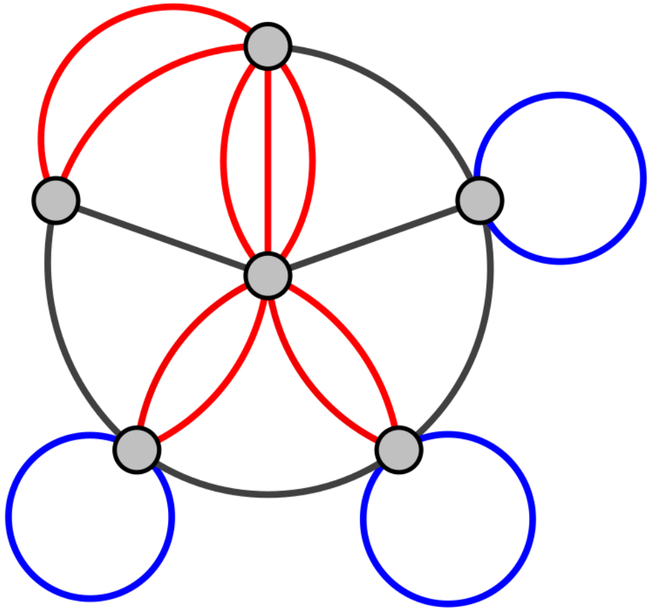
Disconnected Graph
Includes 3 components.

TYPES OF GRAPHS



- **Complete Graph:** A graph that an edges between each pair of distinct vertices.

- **Multigraph Graph:** A graph that allows multiple edges between vertices.



GRAPH AS ADT

- A Graph ADT used to represent **relationships** between objects.
- They can have different variations based on **structure, properties, and constraints**.
- Graphs can be stored and implemented using **arrays** or **linked lists**, depending on the chosen representation.

GRAPH ADT OPERATION

- Basic Operations:
  - **Create an empty graph** – Initializes an empty graph with no vertices or edges.
  - **Add a vertex** – Inserts a new vertex into the graph.
  - **Remove a vertex** – Deletes a vertex and its associated edges.
  - **Add an edge** – Creates a connection between two vertices
  - **Remove an edge** – Deletes an existing edge between two vertices.

GRAPH ADT OPERATION

- Query Operations:
  - **Get all vertices** – Returns a list of all vertices in the graph.
  - **Get all edges** – Returns a list of all edges in the graph.
  - **Check if a vertex exists** – Determines if a specific vertex is present.
  - **Check if an edge exists** – Determines if there is an edge between two vertices.

GRAPH ADT OPERATION

- **Get neighbors of a vertex** – Retrieves all adjacent vertices of a given vertex.
- **Get the degree of a vertex** – Returns the number of edges connected to a vertex.
- **Check if the graph is connected** – Determines if all vertices are reachable from each other.
- **Check if the graph has cycles** – Detects cycles within the graph.

GRAPH ADT OPERATION

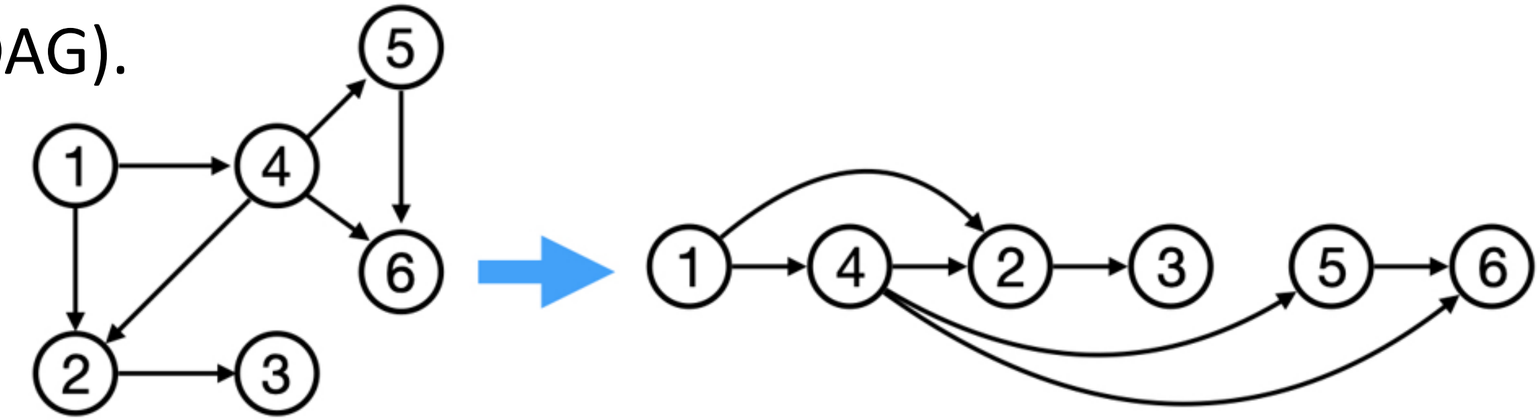
- Traversal Operations:
  - **Perform Depth-First Search (DFS)** – Explores nodes in depth-first order (using a stack).
  - **Perform Breadth-First Search (BFS)** – Explores nodes in breadth-first order (using a queue).

GRAPH ADT OPERATION

- Shortest Path & Search Operations:
  - Find the shortest path using **Dijkstra's algorithm** – Computes the shortest path between two vertices in a weighted graph.
  - Find the shortest path using **Bellman-Ford algorithm** – Computes the shortest path, even with negative weights.
  - Find all-pairs shortest paths using **Floyd-Warshall algorithm** – Computes shortest paths between all vertex pairs.
  - Find shortest path using **A algorithm*** – Uses heuristics for efficient pathfinding.

GRAPH ADT OPERATION

- Special Graph Operations:
  - **Perform topological sorting** – Orders vertices in a directed acyclic graph (DAG).

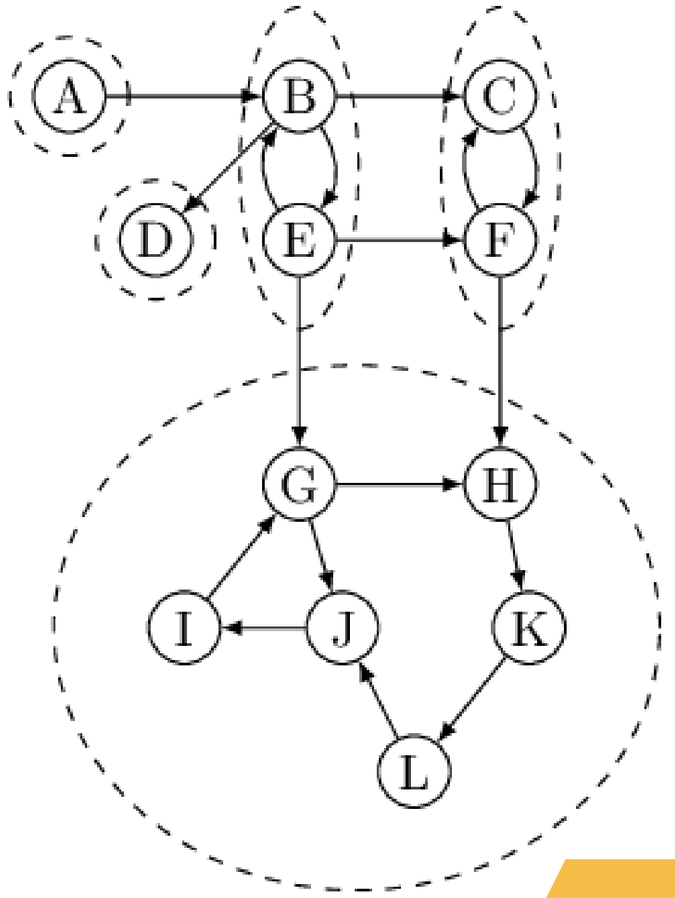


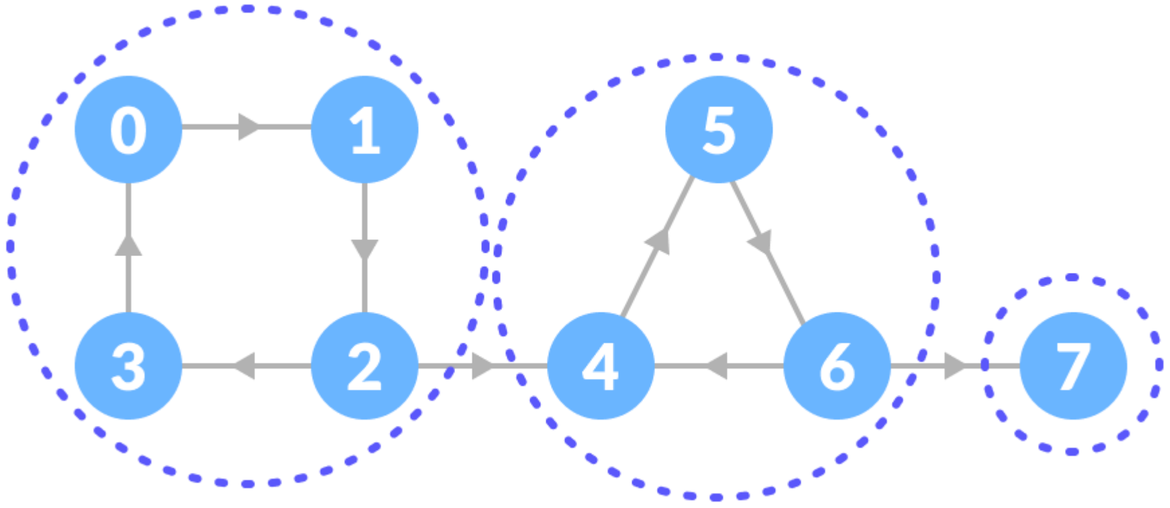
  - It is used for task scheduling, course prerequisites, build systems, etc.

GRAPH ADT OPERATION

- Special Graph Operations:
  - **Find strongly connected components (SCCs)**

    – identifies strongly connected subgraphs.



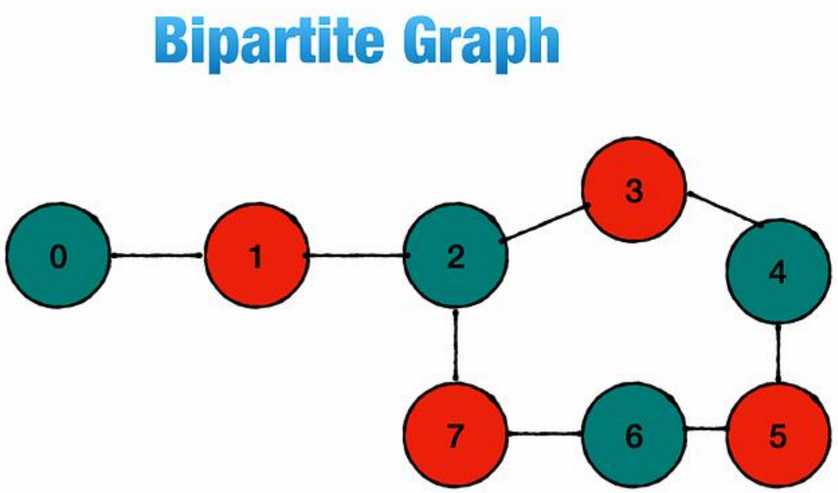


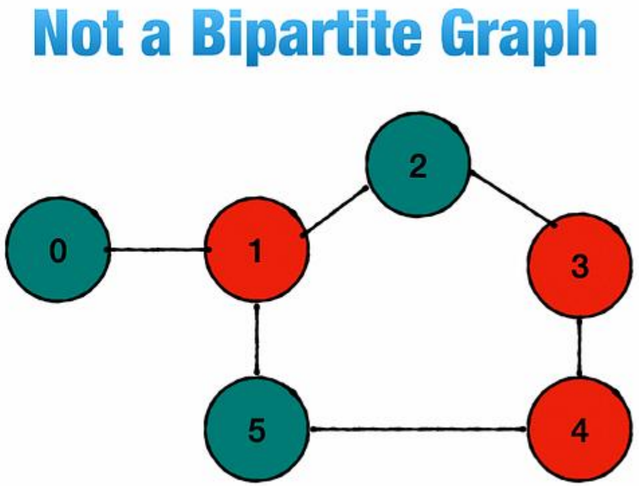
GRAPH ADT OPERATION

- Example of applications:
  - Social networks – Finding groups where all users are interconnected
  - Website crawlers – Identifying clusters of interconnected web pages
  - Deadlock detection – Detecting cycles in resource allocation graphs

GRAPH ADT OPERATION

- Special Graph Operations:
  - **Check if the graph is bipartite** – Determines if the graph can be colored with two colors without adjacent conflicts.





GRAPH ADT OPERATION

- Common applications:
  - Matching Problems (e.g., job assignments, dating apps, team formation)
  - Network Flow (e.g., maximum flow algorithms)
  - Scheduling Problems (e.g., separating tasks into two groups)

GRAPH IMPLEMENTATION

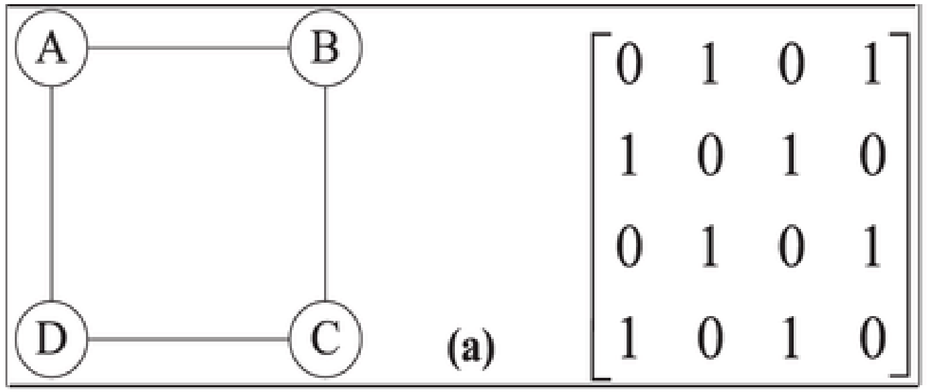
- Graphs can be implemented in multiple ways, depending on the use case and efficiency requirements.
- The most common implementations are:
  - **Adjacency matrix**
  - **Adjacency list**
  - **Edge list representation**

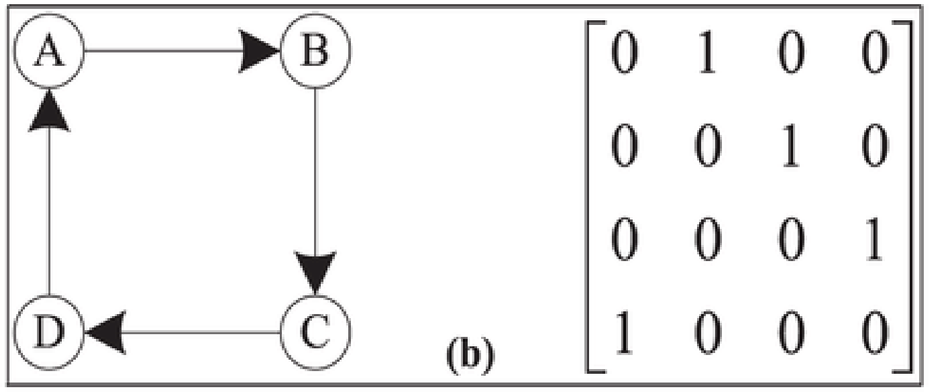
GRAPH IMPLEMENTATION

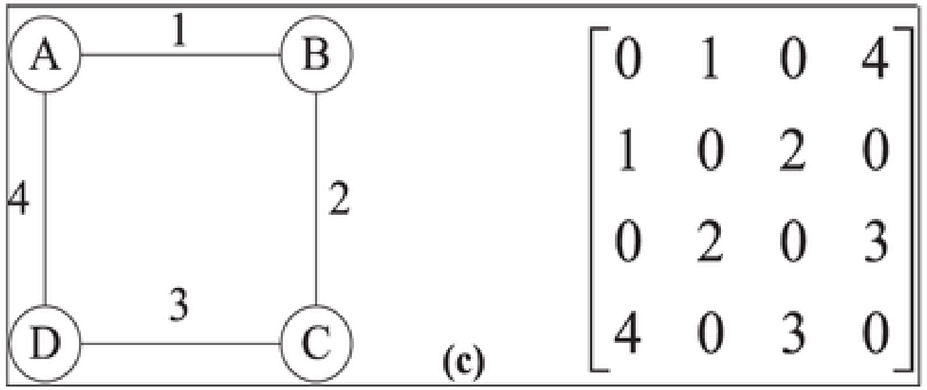
- **Adjacency matrix**
  - Using a 2D array
  - If an edge exists between two vertices, store 1 (or weight in weighted graphs); otherwise, store 0.
  - Suitable for dense graphs (many edges).

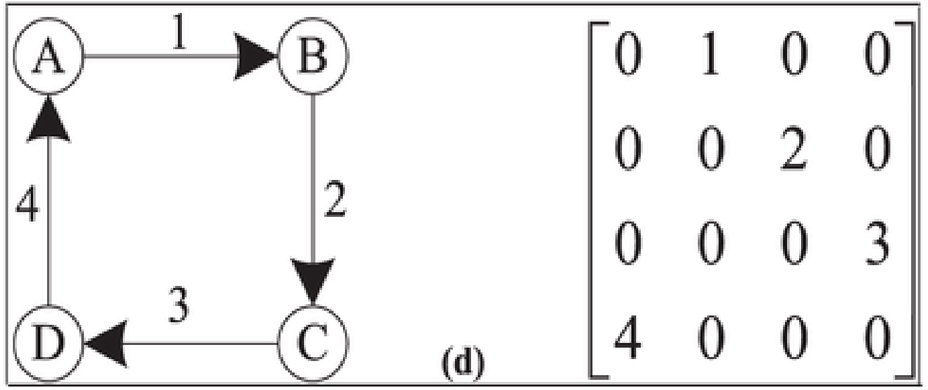
GRAPH IMPLEMENTATION

(a) An unweighted-undirected
(b) An unweighted-directed
(c) A weighted-undirected
(d) a weighted-directed



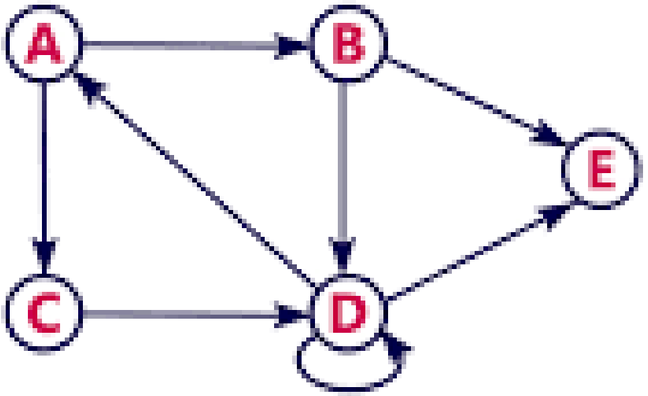


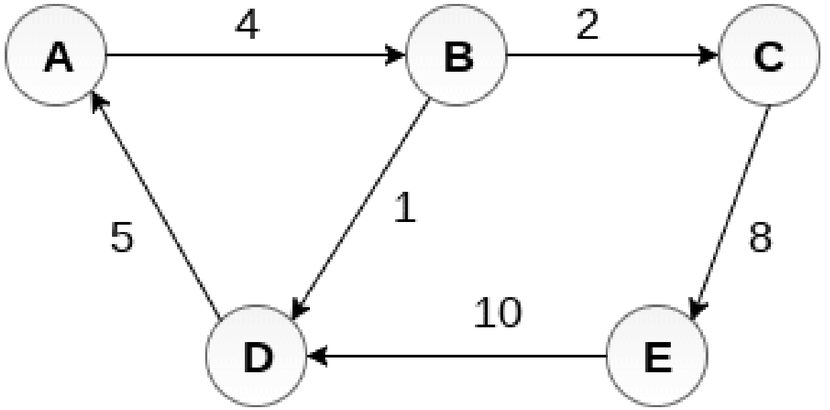




GRAPH IMPLEMENTATION

Test Yourself!



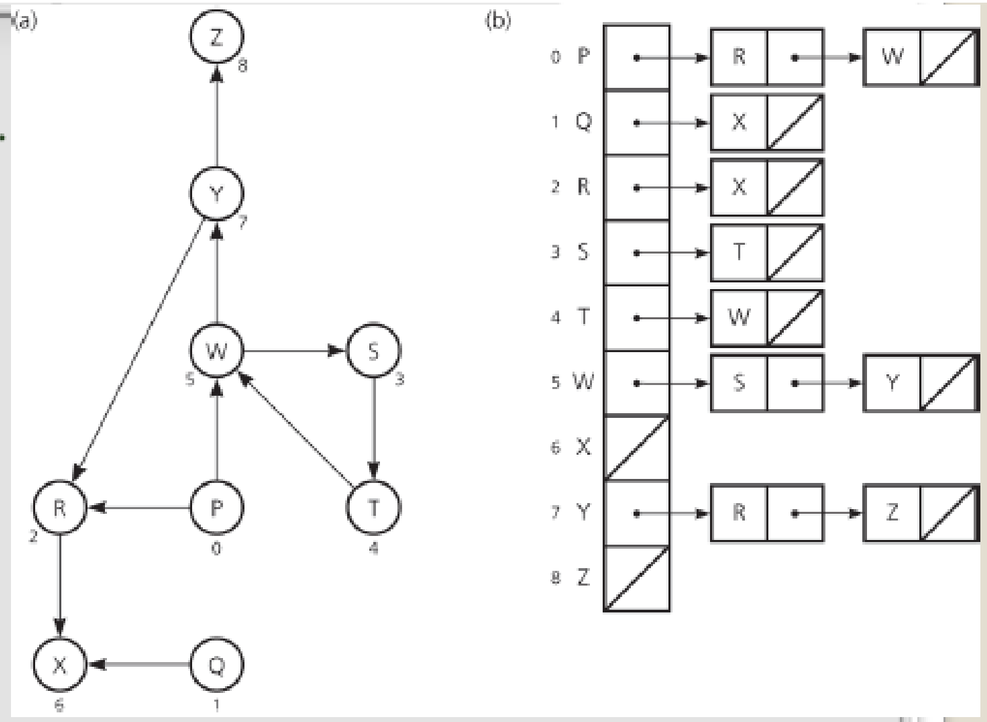


GRAPH IMPLEMENTATION

- **Adjacency list**
  - Using a 2D array
  - If an edge exists between two vertices, store 1 (or weight in weighted graphs); otherwise, store 0.
  - Suitable for dense graphs (many edges).

GRAPH IMPLEMENTATION

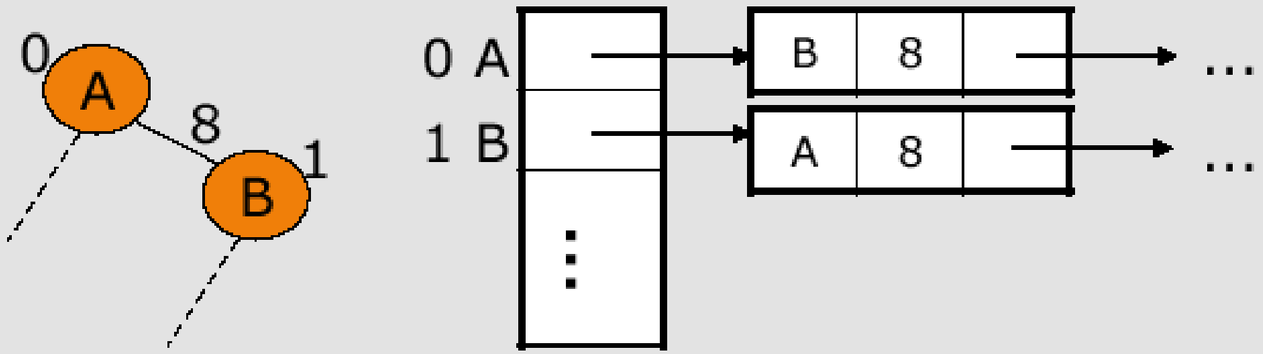
- **Adjacency list** for a directed graph that has n vertices numbered 0, 1, … $n - 1$.
  - An array of n linked lists.
    - Node * list[n];
  - The ith linked list has a node for vertex j if and only if an edge exists from vertex i to vertex j.
  - The list's node can contain:
    - Vertex j's **value**, if any, or an indication of vertex j's identity.
    - A **pointer** points to the next node.
    - **Edge weight**.




```
class Node
{
    String ending_vertex_value;
    float weight;
    Node *next;
}
```

GRAPH IMPLEMENTATION

- For an undirected graph, treat each edge as if there were two directed edges in opposite directions.



- Which of these two implementations is better?
  - Depends on how your application uses the graph.

GRAPH IMPLEMENTATION

- Two common operations on graphs:
  1. Determine whether there is an edge from vertex *i* to vertex *j*.
  2. Find **all** vertices adjacent to a given vertex *i*.

- Adjacency matrix:
  - Supports operation 1 more efficiently.
    - Only need to examine the value of `matrix[i][j]`.

- Adjacency list:
  - Supports operation 2 more efficiently.
    - Traverse the ith linked list.

GRAPH IMPLEMENTATION

- Space requirements of the two implementations:
  - The adjacency matrix always has n^2 entries.
  - The number of nodes in an adjacency list equals the number of edges in a graph (twice for an undirected graph).
    - Each node contains both a value and a pointer.
    - The list also has n head pointer.
    - **Often** requires less space than an adjacency matrix.

- You must consider:
  - What operations are needed.
  - & memory requirements.

GRAPH TRAVERSAL

❖**Graph Traversals**

- Visits all the vertices that it can reach.
  - **Traversal** that starts at vertex *v* will visit all vertices *w* for which there is a path between *v* and *w*.
  - Visits all vertices of the graph if and only if the graph is connected.

- If a graph is **not** connected, a graph traversal will visit only a **subset** of the graph's vertices.
  - A **connected component** : the subset of vertices visited during a traversal that begins at a given vertex.

GRAPH TRAVERSAL

- If a graph contains a cycle, a graph-traversal algorithm can loop indefinitely.

- To prevent indefinite loops:
  - Mark each vertex during a visit, and;
  - Never visit a vertex more than once.

- Two basic graph-traversal algorithms, which are **depth-first search traversal** and **breadth-first search traversal**, can apply to either directed or undirected graphs.

GRAPH TRAVERSAL

❖Depth-First Search (DFS) Traversal

- Proceeds along a path from a vertex *v* as deeply into the graph as possible before backing up.

- Has a simple **recursive** form.

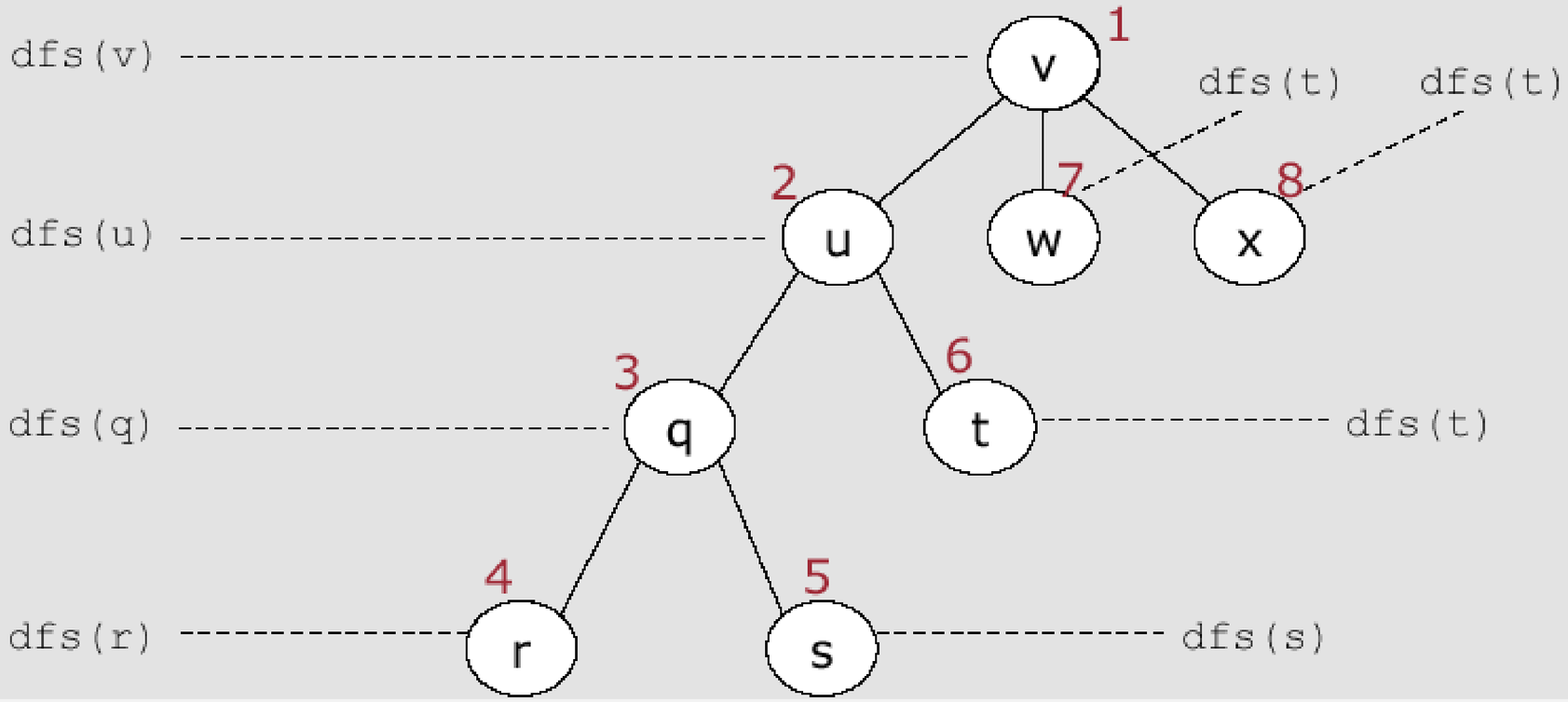
```
dfs(in v:Vertex)
// Traverses a graph beginning at vertex v by using a
// depth-first search: Recursive version.

   Mark v as visited
   for (each unvisited vertex u adjacent to v)
      dfs(u)
```

to visit the vertices adjacent to *v* in sorted order.

GRAPH TRAVERSAL

```
dfs(in v:Vertex)
    Mark v as visited
    for (each unvisited vertex u adjacent to v)
        dfs(u)
```



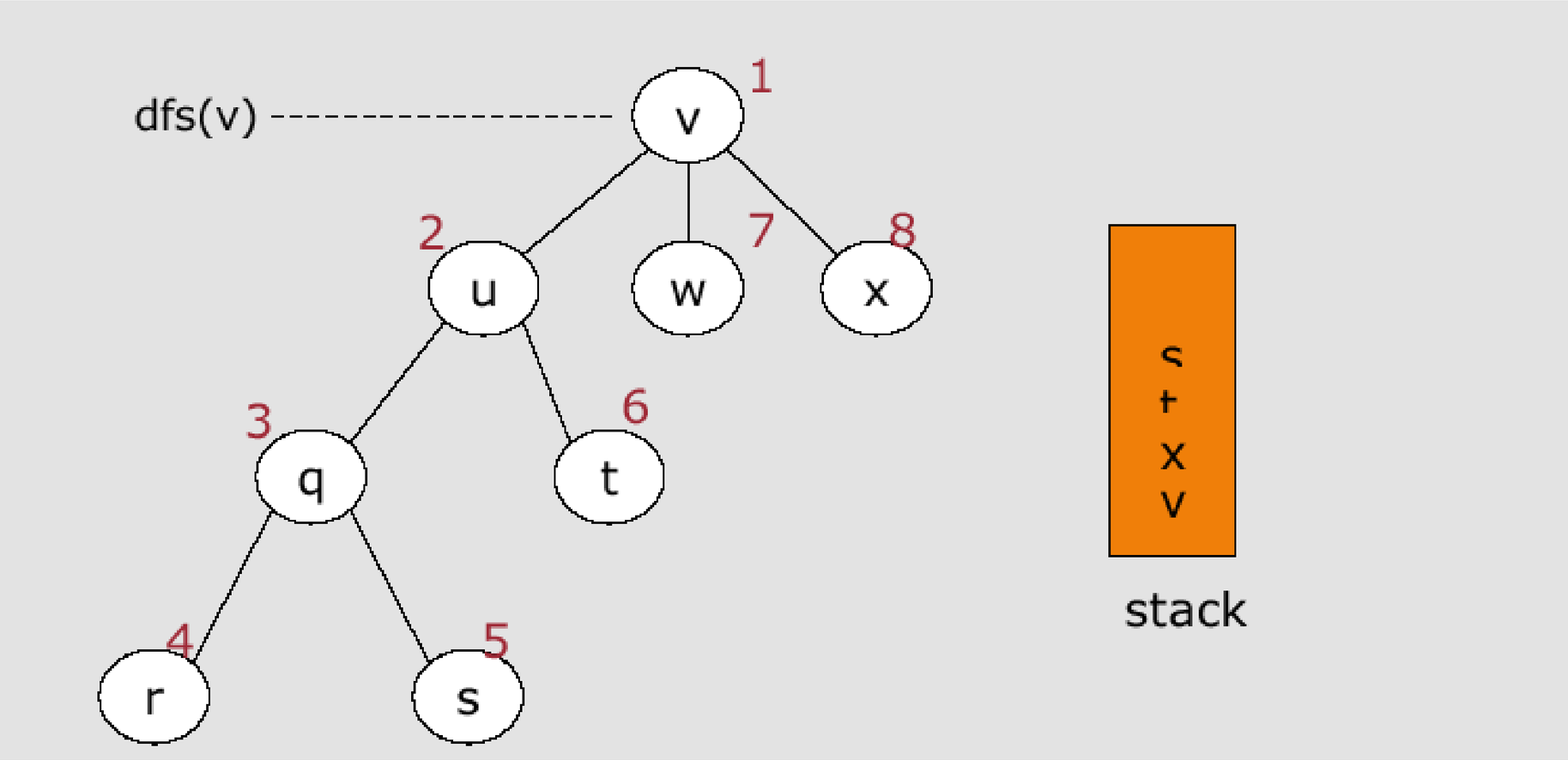
GRAPH TRAVERSAL

- A *"last visited, first explored"* strategy.
  - Has an **iterative** form that uses a **stack**.

```
dfs(in v:Vertex)
   s.createStack()
   s.push(v)
   Mark v as visited

   while(!s.isEmpty())
   {  if (no unvisited vertices are adjacent to
            the vertex on the top of the stack)
         s.pop() // backtrack
      else
      {  Select an unvisited vertex u adjacent to
            the vertex on the top of the stack
         s.push(u)
         Mark u as visited
      }
   }
```

GRAPH TRAVERSAL



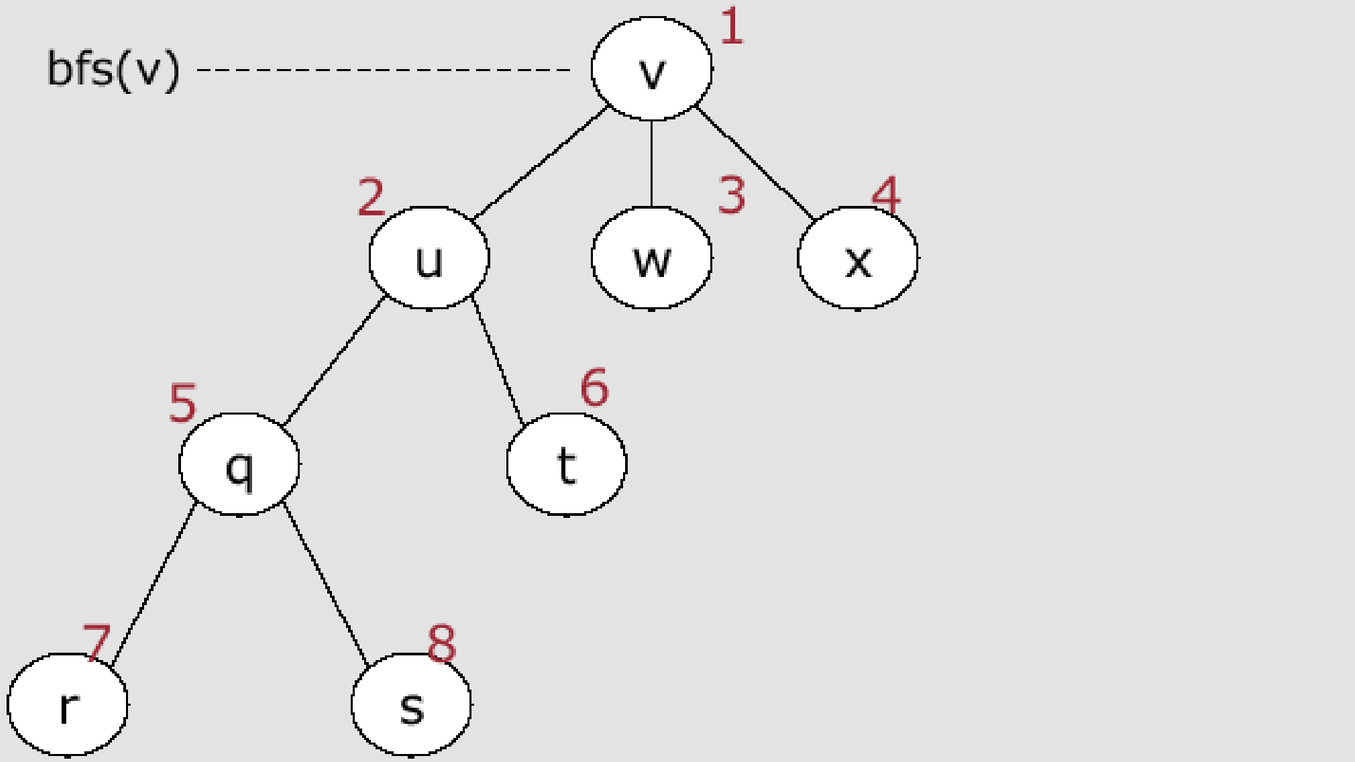
GRAPH TRAVERSAL

❖Breadth-First Search (BFS) Traversal

- Visits **every** vertex adjacent to a vertex *v* that it can before visiting any other vertex.
- A "first visited, first explored" strategy.
- An iterative form uses a **queue**.

```
bfs(in v:Vertex)
    q.createQueue()
    q.enqueue(v)
    Mark v as visited
    while(!q.isEmpty())
    {   q.dequeue(w)
        for (each unvisited vertex u adjacent to w)
        {   Mark u as visited
            q.enqueue(u)
        }
    }
```

GRAPH TRAVERSAL



SUMMARY

- Graphs are fundamental data structures used to represent relationships between objects.

- They consist of **vertices** (or nodes) and **edges** (connections between nodes), allowing for the modeling of complex systems in various fields, including computer science, biology, social networks, and transportation.

- Graphs can be **directed** or **undirected**, weighted or unweighted, depending on whether the relationships have a specific direction or value associated with them.

SUMMARY

- Graph theory encompasses several important concepts, including traversal algorithms such as **Depth-First Search (DFS)** and **Breadth-First Search (BFS),** which are used to explore nodes and edges systematically.

- These algorithms have diverse applications, such as finding the shortest path in routing problems, analyzing social connections in networks, and optimizing resource allocation in logistics.